Lab 3 — Le monolithe modulaire

Des frontières entre les métiers, gardées par un test

Dr. El Hadji Bassirou TOURE
Département de Mathématiques et Informatique
Faculté des Sciences et Techniques
Université Cheikh Anta Diop

Année universitaire 2026–2027

Résumé

L'hexagone du Lab 2 protège votre métier de la technique — mais tous les métiers de SénResto cohabitent dans le même hexagone, libres de s'appeler et de se mélanger. Ce lab trace les frontières manquantes : le domaine est découpé en **cinq modules** (catalogue, commandes, livraison, paiement, notifications) suivant ses *bounded contexts*, chaque membre du groupe devenant **propriétaire d'un module**. **Spring Modulith** transforme ces frontières en propriété vérifiée : un test, `verify()`, casse le build à la moindre violation. Les modules collaborent de deux façons que vous apprendrez à choisir : l'appel direct d'API publique quand il faut une réponse, et les **événements internes** quand on informe — le module `paiement` naîtra d'ailleurs en s'abonnant à un événement, sans modifier une ligne du module qui le publie. Comme toujours : les 8 tests existants restent verts, et le frontend ne remarque rien.

Benefice

À l'issue de ce lab, vous saurez :

- découper un domaine en **bounded contexts** en écoutant où le vocabulaire change de sens ;
- structurer un module Spring Modulith : API publique au paquetage racine, implémentation dans `interne/` ;
- faire garder les frontières **par le build** (`ApplicationModules.verify()`) — et lire ses verdicts ;
- publier et écouter des **événements métier internes**, et choisir entre événement et appel direct.

Point d'attention

Prérequis avant la séance :

- votre dépôt au tag `lab3-depart` (= `lab2-final`, publié à l'ouverture) ;
- `./mvnw test` vert : **8 tests** (4 intégration + 4 unitaires du domaine) ;
- connexion internet en début de séance (nouvelles dépendances Maven : Spring Modulith).

Table des matières

1 Le constat : cartographier les mélanges **3**
1.1 Exercice (15 minutes, en groupe) 3

2 Les concepts : modules et gardien **3**
2.1 La cible : la nouvelle carte des paquetages 4

3 Etape 1 — Monter les murs **5**
3.1 Les dépendances Modulith 5
3.2 Le neuvième juge 5
3.3 Le déménagement 5

4 Etape 2 — Les événements internes **6**
4.1 Commandes publie des faits 6
4.2 Notifications s'abonne — memes SMS qu'avant 7
4.3 Paiement nait d'un evenement 7

5 Etape finale — Eprouver le gardien **8**
5.1 La violation volontaire 8
5.2 La documentation qui ne ment pas 8

6 Recapitulatif : qui a fait quoi **9**

7 L'ADR du jour **9**

8 Livrable et criteres de reussite **9**

9 Vers le Lab 4 : le depart d'un module **10**

1 Le constat : cartographier les mélanges

Intuition

Une grande maison sans murs intérieurs fonctionne très bien... pour une personne. À cinq, chacun a besoin de sa chambre. Votre domaine est cette maison : avant de monter les murs, dessinons le plan — qui vit où, et qui traverse actuellement le salon de qui.

1.1 Exercice (15 minutes, en groupe)

Parcourez `domaine/` et remplissez ce tableau : pour chaque classe, son futur module, et les autres classes qu'elle touche directement.

Classe actuelle	Futur module	Elle touche à...
`CommandeMetier`	commandes	
`RestaurantMetier`	catalogue	
`LivreurMetier`	livraison	
`CalculateurEta`	livraison	
`QuartiersDakar`	? (discutez !)	
`NotificationsPort`	notifications	

Deux découvertes attendues. D'abord, `CommandeMetier` touche à *tout* : plats du catalogue, livreurs, notifications — c'est le carrefour, et c'est normal : commander *orchestre*. Ensuite, le cas `QuartiersDakar` n'a pas de réponse évidente : les frais concernent les commandes, les GPS concernent la livraison. C'est votre première vraie décision de frontière — matière à ADR (notre choix de référence : le module `livraison` possède les quartiers et expose ce dont `commandes` a besoin via son API publique ; d'autres choix se défendent).

2 Les concepts : modules et gardien

Concept fondamental

Bounded context. Une zone du domaine où les mots ont *un* sens précis et le modèle est cohérent. La frontière passe où **le vocabulaire change de sens** : « commande » ne désigne pas la même chose pour Commandes (panier, statut), Livraison (course, ETA) et Paiement (montant à encaisser). Cinq contextes chez SénResto : catalogue, commandes, livraison, paiement, notifications — un par membre.

Concept fondamental

Module Spring Modulith. Chaque paquetage directement sous l'application est un *module*. Son paquetage racine est son **API publique** (types accessibles aux autres modules, événements compris) ; tout sous-paquetage — par convention `interne/` — est **invisible** des autres modules. Et cette règle n'est pas un vœu pieux : le test `ApplicationModules.verify()` casse le build à la moindre violation, fichier et ligne à l'appui.

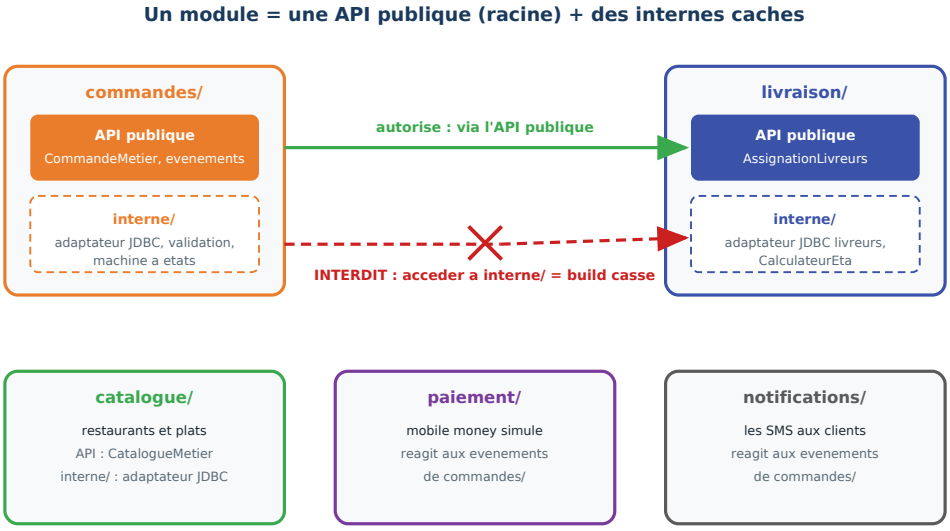


FIGURE 1 – La cible : cinq modules, API publiques et internes cachés.

Concept fondamental

Le noyau partagé (shared kernel). Certains types servent plusieurs modules — ici, les exceptions métier que `rest` traduit en HTTP et que `catalogue` comme `commandes` lèvent. Les loger dans `commandes` créerait `catalogue` → `commandes` → `catalogue` : un **cycle**, que `verify()` refuse. Le remède DDD : un petit module `partage`, dont tout le monde peut dépendre et qui ne dépend de personne. Anecdote véridique : ce cycle a été *réellement détecté par `verify()`* pendant la conception de ce corrigé — le gardien ne fait pas de politesse, même à l'enseignant.

2.1 La cible : la nouvelle carte des paquetages

```
sn/ucad/senresto/
|-- SenRestoApplication.java
|-- catalogue/
|   |-- CatalogueMetier.java              <- API publique
|   \-- interne/
|       |-- StockageCatalogue.java  (interface, pour les tests in-memory)
|       |-- AdaptateurCatalogueJdbc.java
|       \-- ConfigurationCatalogue.java
|-- commandes/
|   |-- CommandeMetier.java               <- API publique
|   |-- CommandeRequete.java  ChangementStatutRequete.java
|   |-- CommandeCreee.java  CommandeConfirmee.java  CommandeStatutChangee.java
|   |-- PublicationEvenements.java
|   \-- interne/
|       |-- StockageCommandes.java
|       |-- AdaptateurCommandesJdbc.java
|       |-- PublicationEvenementsSpring.java
|       \-- ConfigurationCommandes.java
|-- livraison/
|   |-- AssignationLivreurs.java  Assignation.java   <- API publique
|   |-- QuartiersDakar.java
|   \-- interne/
|       |-- StockageLivreurs.java
|       |-- AdaptateurLivreursJdbc.java
|       |-- CalculateurEta.java
|       \-- ConfigurationLivraison.java
|-- paiement/
|   \-- interne/
|       \-- EcouteurPaiement.java         <- nait d'un evenement
|-- partage/                              <- le noyau partage (shared kernel)
|   |-- ValidationException.java  IntrouvableException.java
|   \-- ConflitException.java
|-- notifications/
|   \-- interne/
|       |-- EcouteurNotifications.java
|       \-- AdaptateurSmsConsole.java
```

```
\-- rest/
    |-- SenRestoController.java  GestionnaireErreurs.java
```

Point d'attention

Le lab conserve l'esprit hexagonal *à l'intérieur* de chaque module (le métier au centre, les adaptateurs en `interne/`) mais simplifie le cérémonial : les ports explicites du Lab 2 ne sont conservés que là où ils servent encore (tests in-memory de `commandes`). C'est un choix d'équilibre assumé — discutez-le, c'est une excellente ligne d'ADR.

3 Etape 1 — Monter les murs

Qui pilote cette section ?

Chaque membre devient **propriétaire d'un module** : le Doyen — `commandes` ; l'Huissier — `livraison` ; le Rédacteur — `catalogue` ; le Notaire — `notifications` ; l'Appariteur — `paiement` (qui attendra l'étape 2 : son module naît des événements). À l'étape 1, chacun déménage ses classes chez lui.

3.1 Les dépendances Modulith

Dans le `pom.xml`, ajoutez le BOM et les deux artefacts :

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.modulith</groupId>
      <artifactId>spring-modulith-bom</artifactId>
      <version>1.2.5</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

```
<dependency>
  <groupId>org.springframework.modulith</groupId>
  <artifactId>spring-modulith-starter-core</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.modulith</groupId>
  <artifactId>spring-modulith-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

3.2 Le neuvième juge

Avant même de déménager, installez le gardien — il sera rouge, c'est normal, et vous le regarderez passer au vert module par module :

```
package sn.ucad.senresto;

import org.junit.jupiter.api.Test;
import org.springframework.modulith.core.ApplicationModules;

class ModulithArchitectureTest {

    @Test
    void lesFrontieresDesModulesSontRespectees() {
        ApplicationModules modules = ApplicationModules.of(SenRestoApplication.class);
        modules.forEach(System.out::println);   // la carte, lue depuis le code
        modules.verify();                       // le verdict
    }
}
```

3.3 Le déménagement

Chaque propriétaire crée son paquetage et son `interne/`, y déplace ses classes (l'IDE fait les imports), en appliquant la règle : *ce que les autres modules utilisent* reste à la racine, tout le reste descend dans `interne/`. Cas guidés :

— **livraison** expose `AssignationLivreurs` et le record `Assignation`. Attention à la conception de l'API : pas de méthodes `premierDisponible` puis `reserver` séparées, mais une seule opération `assigner(quartier)` qui choisit, calcule l'ETA et réserve. Pourquoi ? Parce que `CalculateurEta` descend en `interne/` : si `commandes` devait calculer l'ETA lui-même, il lui faudrait ce secret — violation immédiate. *L'ETA est un secret de livraison* : c'est l'encapsulation qui dessine l'API ;
— **commandes** garde à sa racine `CommandeMetier`, les deux requêtes et les trois exceptions (le module `rest` les traduit en HTTP) ;
— **catalogue** expose `CatalogueMetier` (ex-`RestaurantMetier`, renommé au passage : le vocabulaire du contexte) ;
— `CommandeMetier` appelle désormais `AssignationLivreurs` et `CatalogueMetier` — des API publiques : autorisé, et `verify()` l'atteste ;
— la `ConfigurationDomaine` globale du Lab 2 disparaît : **chaque module assemble son propre métier** dans une `@Configuration` de son `interne/` — les murs valent aussi pour l'assemblage.

Point d'attention

Le piège des classes fantômes. Après un grand déménagement de paquetages, lancez toujours `./mvnw clean test` — pas `test` seul. Maven compile les nouvelles sources mais ne supprime jamais les `.class` des classes disparues : `verify()` et Spring, qui lisent `target/classes`, verraient alors les *fantômes* de l'ancienne architecture (violations inexplicables, doublons de beans). Vécu et diagnostiqué sur ce cours même.

Tache a realiser

À réaliser — étape 1 (une branche `feature/module-xxx` par propriétaire) : les cinq paquetages montés (paiement encore vide, avec un `package-info.java`), toutes les classes déménagées, le `ModulithArchitectureTest` **vert**, les 8 tests existants **verts**, le frontend intact.

Jalon Git

9 tests verts (8 + verify) :

```
git tag lab3-etape1 && git push origin lab3-etape1
```

4 Etape 2 — Les événements internes

Qui pilote cette section ?

Le Doyen fait publier `commandes`. **Le Notaire** abonne `notifications`. **L'Appariteur** fait naître `paiement`. **L'Huissier** et **le Rédacteur** vérifient que leurs modules n'ont *rien* à changer — c'est aussi une leçon.

4.1 Commandes publie des faits

Trois records dans la racine de `commandes` — des faits passés, nommés au participe :

```
public record CommandeCreee(long commandeId, String numero,
        String clientTelephone, String restaurantNom, int montantTotal) {}

public record CommandeConfirmee(long commandeId, String numero,
        String clientTelephone, int montantTotal,
        String livreurNom, int etaMinutes) {}

public record CommandeStatutChangee(long commandeId, String numero,
        String clientTelephone, String nouveauStatut) {}
```

Pour rester fidèle à l'hexagone, `CommandeMetier` ne dépend pas de Spring : il publie via une petite interface de sa racine, dont l'implémentation Spring vit en `interne/` :

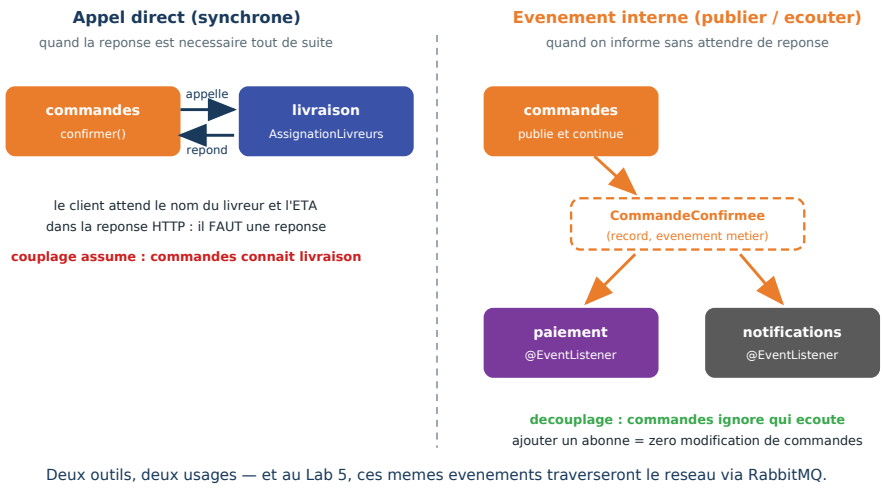


FIGURE 2 – Appel direct quand il faut une réponse ; événement quand on informe.

```java
// commandes/PublicationEvenements.java  (racine)
public interface PublicationEvenements {
    void publier(Object evenement);
}

// commandes/interne/PublicationEvenementsSpring.java
@Component
class PublicationEvenementsSpring implements PublicationEvenements {
    private final ApplicationEventPublisher spring;
    PublicationEvenementsSpring(ApplicationEventPublisher spring) { this.spring = spring; }
    @Override public void publier(Object evenement) { spring.publishEvent(evenement); }
}
```

Dans `CommandeMetier`, les appels directs aux notifications sont remplacés par des publications — et la dépendance aux notifications **disparaît** de son constructeur :

```java
// creer() : a la place de notifications.envoyerSms(...)
evenements.publier(new CommandeCreee(id, numero, clientTelephone,
        (String) plat.get("restaurant_nom"), montantTotal));
```

4.2 Notifications s'abonne — memes SMS qu'avant

```java
// notifications/interne/EcouteurNotifications.java
@Component
class EcouteurNotifications {

    private final AdaptateurSmsConsole sms;

    EcouteurNotifications(AdaptateurSmsConsole sms) { this.sms = sms; }

    @EventListener
    void quandCommandeCreee(CommandeCreee evt) {
        sms.envoyerSms(evt.clientTelephone(), "Commande " + evt.numero()
                + " recue chez " + evt.restaurantNom()
                + " -- total " + evt.montantTotal() + " FCFA");
    }
    // quandCommandeConfirmee(...), quandStatutChange(...) : meme principe,
    // MEMES textes de SMS qu'au depart -- le comportement observable
    // ne change pas (les evenements Spring sont synchrones par defaut).
}
```

4.3 Paiement nait d'un evenement

Le moment pédagogique du lab : l'Appariteur crée le module `paiement` **sans qu'aucun autre module ne soit modifié** — la preuve vivante du découplage :

```java
// paiement/interne/EcouteurPaiement.java
@Component
class EcouteurPaiement {

    @EventListener
    void quandCommandeConfirmee(CommandeConfirmee evt) {
```

```
        System.out.println("[PAIEMENT] Encaissement mobile money simule : "
                + evt.montantTotal() + " FCFA pour la commande " + evt.numero());
    }
}
```

Exemple

Passez une commande depuis le frontend et confirmez-la : dans la console, le `[SMS]` habituel est maintenant suivi d'une ligne `[PAIEMENT]`. Deux modules ont réagi au même fait — et `commandes` l'ignore superbement. Au Lab 5, Wave et Orange Money « réels » (simulés) remplaceront ce `System.out`, avec webhooks et idempotence.

Point d'attention

Les tests unitaires de `CommandeMetier` évoluent : le port notifications disparaît de son constructeur au profit de `PublicationEvenements`. En test, un enregistreur suffit — `evenements.add(evt)` dans une liste — et les assertions deviennent plus expressives encore : *vérifier qu'un fait a été publié* plutôt qu'un SMS envoyé.

Tache a realiser

À réaliser — étape 2 :

- Le Doyen : les 3 records, `PublicationEvenements` + son adaptateur Spring, `CommandeMetier` qui publie ;
- Le Notaire : `EcouteurNotifications`, 3 écouteurs, **mêmes textes de SMS** (les tests d'intégration y veillent) ;
- L'Appariteur : le module `paiement` complet ;
- L'Huissier et le Rédacteur : adaptation des tests unitaires de `commandes` (enregistreur d'événements), et vérification que `livraison` et `catalogue` n'ont pas bougé ;
- Tous : `./mvnw test` — 9 juges verts.

Jalon Git

```
git tag lab3-etape2 && git push origin lab3-etape2
```

5 Etape finale — Eprouver le gardien

Qui pilote cette section ?

Tous ensemble, au vidéoprojecteur du groupe : cette étape est une expérience, pas une production.

5.1 La violation volontaire

Dans `CommandeMetier`, ajoutez volontairement un import interdit :

```
import sn.ucad.senresto.livraison.interne.CalculateurEta;   // INTERDIT !
```

Utilisez-le n'importe où, lancez `./mvnw test`, et **lisez le verdict** de `verify()` : le module fautif, le type visé, la violation nommée. Puis retirez l'import. Vous venez de voir la différence entre une convention (qui s'érode) et une architecture (qui casse le build). Recommencez si un membre du groupe est sceptique — c'est fait pour ça.

5.2 La documentation qui ne ment pas

Ajoutez au `ModulithArchitectureTest` :

```
@Test
void genererLaDocumentationDesModules() {
    new Documenter(ApplicationModules.of(SenRestoApplication.class))
```

```
            .writeModulesAsPlantUml()
            .writeIndividualModulesAsPlantUml();
}
```

Les diagrammes apparaissent dans `target/spring-modulith-docs/` : une carte des modules **générée depuis le code**, donc toujours vraie. Le Rédacteur la joint au dépôt (`docs/modules/`) — c'est la pièce jointe rêvée de l'ADR.

Tache a realiser

À réaliser — étape finale : l'expérience de violation faite et défaite (elle doit apparaître dans l'historique Git : le commit qui casse, le commit qui répare — c'est votre preuve de manipulation) ; la documentation générée et versionnée ; les 9 juges verts ; le parcours frontend complet avec la ligne `[PAIEMENT]` dans la console.

Jalon Git

```
git tag lab3-final && git push origin lab3-final
```

6 Recapitulatif : qui a fait quoi

Rôle	Étape 1	Étape 2	Étape finale
Le Doyen	module `commandes`	records + publications	tag final, ADR
L'Huissier	module `livraison`, API `AssignationLivreurs`	vérif. « rien à changer » + tests unitaires	violation volontaire
Le Rédacteur	module `catalogue`	vérif. + tests unitaires	documentation générée
Le Notaire	module `notifications`	les 3 écouteurs	relecture des SMS
L'Appariteur	`paiement` (coquille)	le module `paiement` né d'un événement	démo `[PAIEMENT]`

7 L'ADR du jour

L'**Architecte du jour** signe l'**ADR n°3 : « Comment tracer les frontières des modules ? »** (`docs/adr/adr-003-frontieres.md`). Matière obligatoire : la décision `QuartiersDakar` (qui possède les quartiers, et pourquoi) ; le choix appel direct pour `livraison` *versus* événement pour `paiement`/`notifications` (pourquoi pas tout en événements ?). Alternatives sérieuses possibles : loger les exceptions dans chaque module (duplication assumée) plutôt que dans `partage` ; tout-événements avec enrichissement ; garder quatre modules sans `paiement`. Conséquences négatives à assumer : la discipline des paquetages, les API publiques à faire vivre, le risque de modules anémiques.

8 Livrable et criteres de reussite

Critère	Vérification
Les 4 tags du lab présents	`git tag`
9 juges verts sur `lab3-final` (8 tests + verify)	`./mvnw test`
Aucun import d'un `interne/` étranger	`verify()` + relecture
La violation volontaire dans l'historique (cassée puis réparée)	`git log`
Le module `paiement` né sans toucher `commandes`	`git diff` ciblé
Mêmes SMS qu'au départ, ligne `[PAIEMENT]` en plus	console
Documentation des modules versionnée	`docs/modules/`
ADR n°3 signé	`docs/adr/`

9 Vers le Lab 4 : le depart d'un module

Votre monolithe est modulaire : frontières nettes, gardien vigilant, collaboration par API et par événements. C'est exactement pour cette raison que la suite devient *possible* : la direction veut faire évoluer le calcul d'itinéraires avec l'écosystème Python (géospatial, data), et le déployer sans redéployer SénResto. Le module `livraison` — frontières déjà tracées, API déjà publique — est le candidat parfait à l'**extraction**. Au Lab 4 : le pattern Strangler Fig, un microservice FastAPI, une carte Leaflet de Dakar... et une panne instructive dont vous vous souviendrez. Votre `lab3-final` en sera le point de départ.

Fin du Lab 3 — Dr. El Hadji Bassirou TOURE — DMI/FST/UCAD